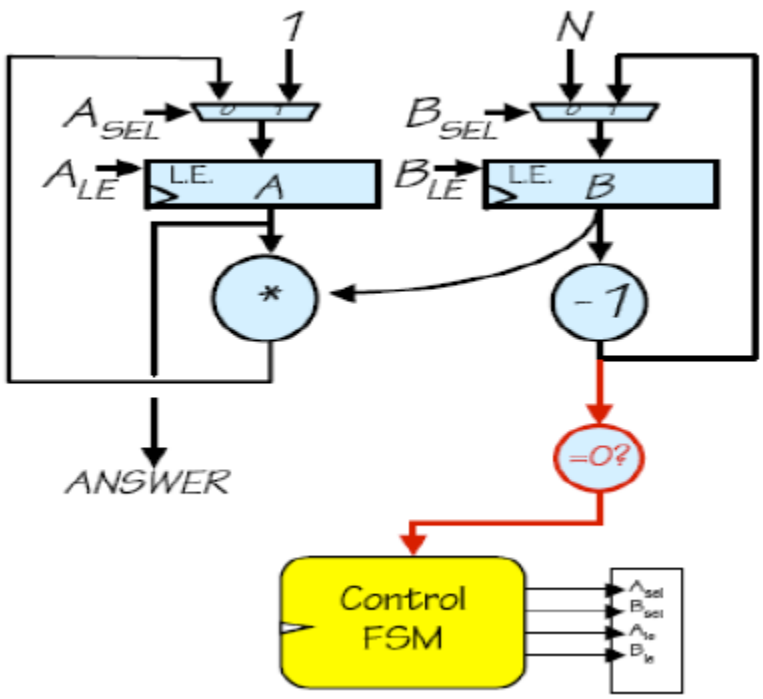
LAB 11

A Taste of Data path + Control Design Example: Factorial Circuit (Open Ended Lab)

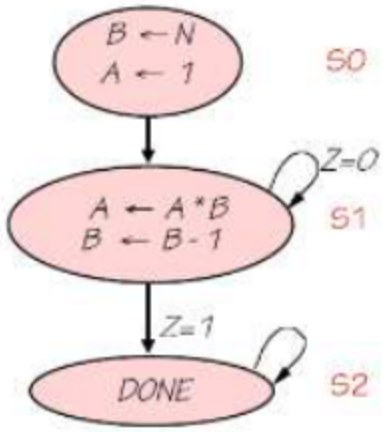
Objective :

To implement a circuit that calculates factorial of a number.

Block Diagram:
The datapath for calculating the factorial of an "N" bit number is given below. The datapath is controlled by a Finite State Machine (FSM). FSM generates signals Asel, Ale, Bsel and Ble at the correct times to operate the datapath. Input to FSM is a signal "Z" when Z=0 means the operation of the machine is complete and ANSWER can be read. The STG is also given in the following diagram.



State Transition Diagram:



I/O Connection:

Connect the input N to switches so automatically N=8 bits and connect the ANSWER to 8 LEDs. Don't give input greater than 5 else the ANSWER will not 8 bit wide.

CODE:

DATAPATH

```
module FactorialDataPath(
    input wire clk,
    input wire reset,
    input wire [3:0] A_input,
    input wire load_A,
    input wire start_calc,
    output reg [7:0] result_out,
    output reg done
);

    reg [3:0] A_reg, B_reg;
    reg [7:0] factorial;

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            A_reg <= 0;
```

```
        B_reg <= 0;
        factorial <= 1;
        result_out <= 0;
        done <= 0;
      end else begin
        if (load_A) begin
          A_reg <= A_input;
          B_reg <= A_input;
          factorial <= 1;
          done <= 0;
        end else if (start_calc && B_reg > 1) begin
          factorial <= factorial * B_reg;
          B_reg <= B_reg - 1;
        end else if (start_calc && B_reg <= 1 && done == 0) begin
          result_out <= factorial;
          done <= 1;
        end
      end
    end

endmodule
```

FSM Controller

```
module FactorialController(
    input wire clk,
    input wire reset,
    input wire datapath_done,
    input wire start,
    output reg load_A,
```

```
    output reg start_calc,
    output reg done
);

    typedef enum reg [1:0] {IDLE = 2'b00, LOAD = 2'b01, CALC = 2'b10, DONE = 2'b11}
state_t;
    state_t state, next;

    always @(posedge clk or posedge reset) begin
        if (reset)
            state <= IDLE;
        else
            state <= next;
    end

    always @(*) begin
        // Default signal values
        load_A = 0;
        start_calc = 0;
        done = 0;

        case (state)
            IDLE: begin
                if (start) next = LOAD;
                else next = IDLE;
            end

            LOAD: begin
                load_A = 1;
```

```
            next = CALC;
        end

        CALC: begin
            start_calc = 1;
            if (datapath_done)
                next = DONE;
            else
                next = CALC;
        end

        DONE: begin
            done = 1;
            next = DONE;
        end

        default: next = IDLE;
    endcase
end

endmodule
```

Top Module

```
module FactorialTop(
    input wire clk,
    input wire reset,
    input wire start,
    input wire [3:0] number,
    output wire [7:0] result,
```

```
    output wire done
);

    wire load_A, start_calc, datapath_done;

    FactorialDataPath datapath_inst (
        .clk(clk),
        .reset(reset),
        .A_input(number),
        .load_A(load_A),
        .start_calc(start_calc),
        .result_out(result),
        .done(datapath_done)
    );

    FactorialController fsm_inst (
        .clk(clk),
        .reset(reset),
        .start(start),
        .datapath_done(datapath_done),
        .load_A(load_A),
        .start_calc(start_calc),
        .done(done)
    );

endmodule
```

Task 2: Implement Multiplication for repeated addition using data path and control path (FSM)
Note: you can do simulation as well
CODE:
DATAPATH

```
module MultiplierDataPath(
    input wire clk,
    input wire reset,
    input wire [3:0] A_input,
    input wire [3:0] B_input,
    input wire load_inputs,
    input wire add_enable,
    output reg [7:0] result_out,
    output reg done
);

    reg [3:0] A_reg, B_reg;
    reg [7:0] sum;

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            A_reg <= 0;
            B_reg <= 0;
            sum <= 0;
            result_out <= 0;
            done <= 0;
        end else begin
            if (load_inputs) begin
                A_reg <= A_input;
                B_reg <= B_input;
                sum <= 0;
                done <= 0;
            end else if (add_enable && B_reg > 0) begin
```

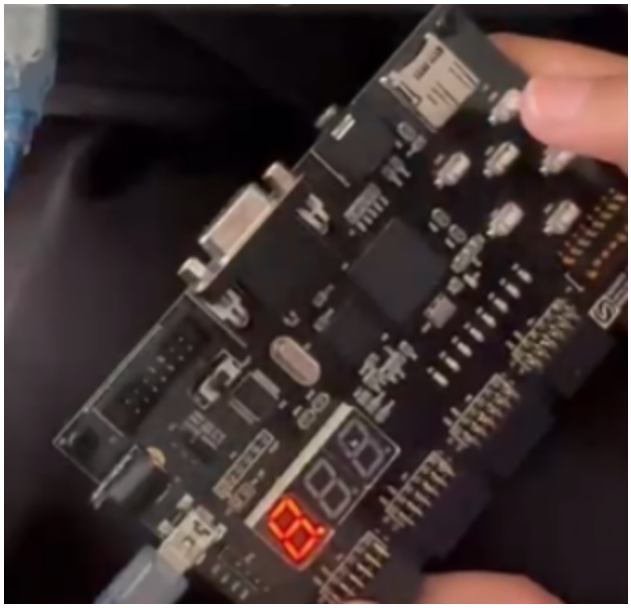
```
            sum <= sum + A_reg;
            B_reg <= B_reg - 1;
        end else if (B_reg == 0 && done == 0) begin
            result_out <= sum;
            done <= 1;
        end
    end
end

endmodule
```

FSM Controller

```
module MultiplierController(
    input wire clk,
    input wire reset,
    input wire start,
    input wire datapath_done,
    output reg load_inputs,
    output reg add_enable,
    output reg done
);

    typedef enum reg [1:0] {IDLE = 2'b00, LOAD = 2'b01, CALC = 2'b10, DONE = 2'b11}
state_t;
    state_t state, next;

    always @(posedge clk or posedge reset) begin
        if (reset)
            state <= IDLE;
        else
            state <= next;
    end

    always @(*) begin
        // Default outputs
        load_inputs = 0;
        add_enable = 0;
        done = 0;

        case (state)
            IDLE: begin
                if (start) next = LOAD;
                else next = IDLE;
            end
```

```
        LOAD: begin
            load_inputs = 1;
            next = CALC;
        end

        CALC: begin
            add_enable = 1;
            if (datapath_done)
                next = DONE;
            else
                next = CALC;
        end

        DONE: begin
            done = 1;
            next = DONE;
        end

        default: next = IDLE;
    endcase
end

endmodule
```

Top Module

```
module MultiplierTop(

    input wire clk,

    input wire reset,

    input wire start,

    input wire [3:0] A,

    input wire [3:0] B,

    output wire [7:0] result,

    output wire done

);


    wire load_inputs, add_enable, datapath_done;
```

```
    MultiplierDataPath datapath_inst (
        .clk(clk),
        .reset(reset),
        .A_input(A),
        .B_input(B),
        .load_inputs(load_inputs),
        .add_enable(add_enable),
        .result_out(result),
        .done(datapath_done)
    );

    MultiplierController fsm_inst (
        .clk(clk),
        .reset(reset),
        .start(start),
        .datapath_done(datapath_done),
        .load_inputs(load_inputs),
        .add_enable(add_enable),
        .done(done)
    );

endmodule
```